MovieLens Rating Prediction

Cristiano Camilo de Araujo Lelis

2026-10-14

Contents

1 Introduction **1**

2 Methods and Analysis **2**

2.1 Data preparation 2

2.2 Data exploration 2

2.3 Modeling approach 5

3 Results **7**

4 Conclusion **7**

5 References **8**

1 Introduction

Recommendation systems are tools that try to predict which items each user is likely to enjoy, based on the user's previous ratings and on the ratings of other users. They are useful because they make the experience more personalized and help users discover new movies, even when the data is sparse, as in this case, where most users have rated only a small fraction of the available titles.

This project uses the MovieLens 10M dataset, which contains about 10 million movie ratings. After 10% of the data was reserved for the final evaluation, the `edx` set used for development contains 9,000,055 ratings given by 69,878 users to 10,677 movies, with an average rating of 3.51. Although this volume of data is large, it represents only about 1.2% of the user-movie matrix: if every user had rated every movie, there would be more than 746 million ratings. This incompleteness is why models that can predict the missing ratings are needed.

The goal of the project was to predict the ratings that users would give to movies and to measure the accuracy of these predictions with the Root Mean Squared Error (RMSE), with a target below 0.8649. The key steps were: data preparation, data exploration, step-by-step construction of bias models, regularization, matrix factorization of the residuals and a final evaluation on the reserved `final_holdout_test` set. The final model reached an RMSE of 0.78655, well below the target.

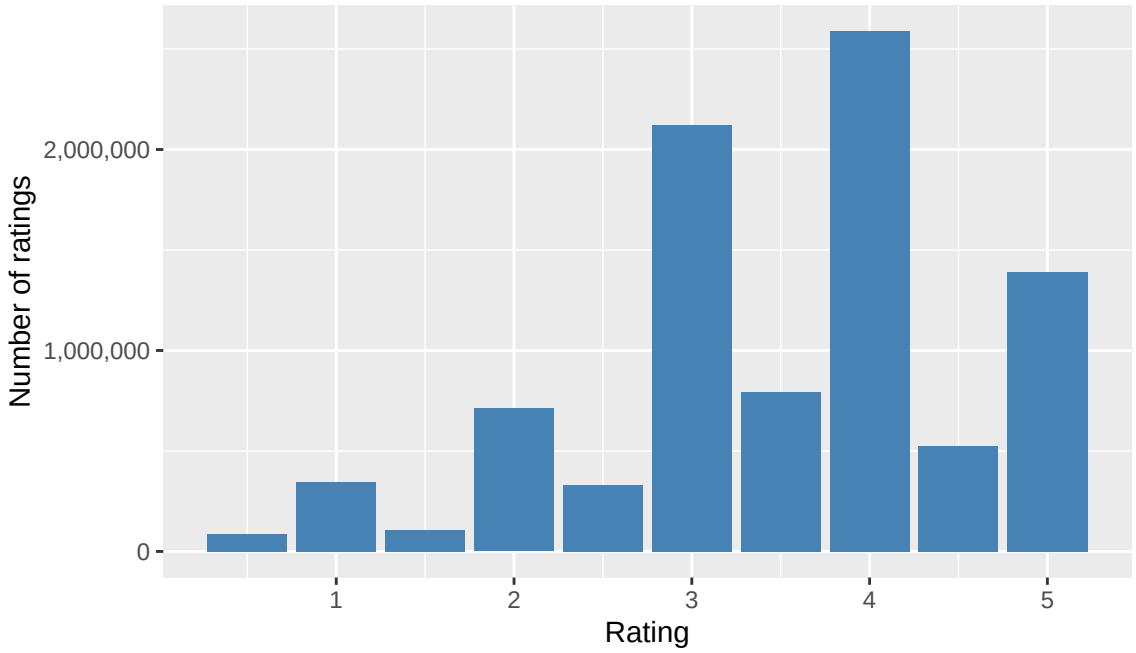
2 Methods and Analysis

2.1 Data preparation

The `edx` and `final_holdout_test` sets were created with the code provided by the HarvardX PH125.9x course. The `final_holdout_test` set corresponds to 10% of the data and was used only once, at the end, to obtain an unbiased estimate of the final model's performance. The `edx` set was split into a `train_set` (90%) and a `test_set` (10%), used to develop, compare and tune the models. As in the course code, users and movies that appear in the `test_set` but not in the `train_set` were moved back to the `train_set`, so that every prediction can use an estimated movie and user effect.

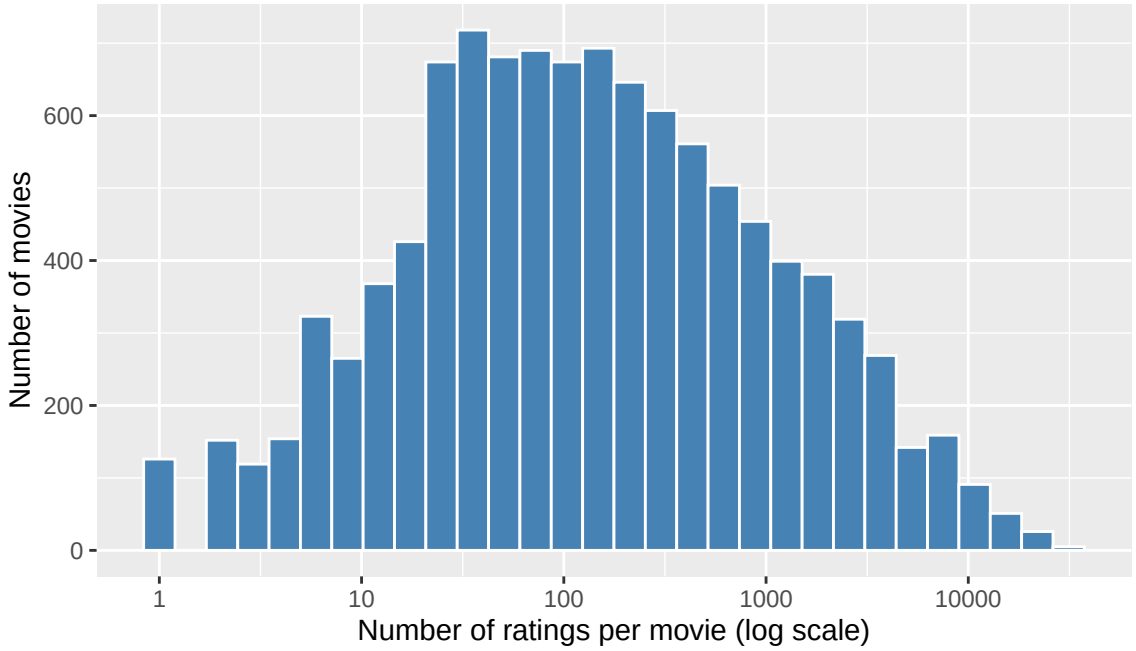
2.2 Data exploration

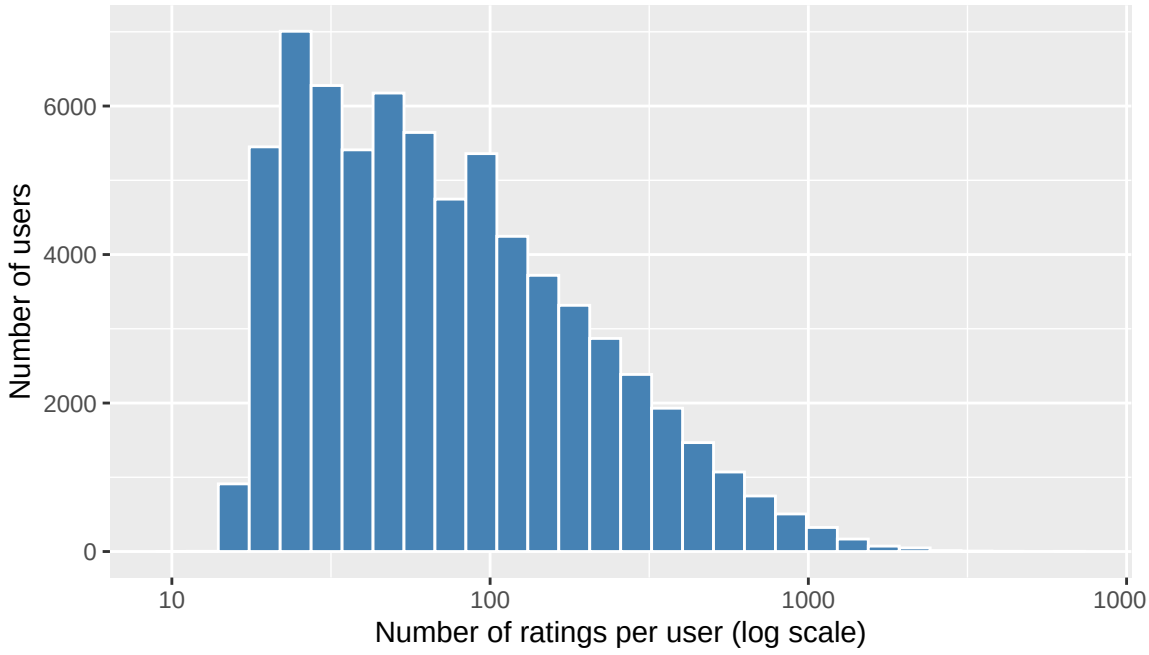
2.2.1 Distribution of ratings



The distribution shows a predominance of whole-star ratings, especially 4, followed by 3 and 5. Low ratings are rare, and the overall average (3.51) is above the midpoint of the scale. This indicates that users tend to rate movies positively.

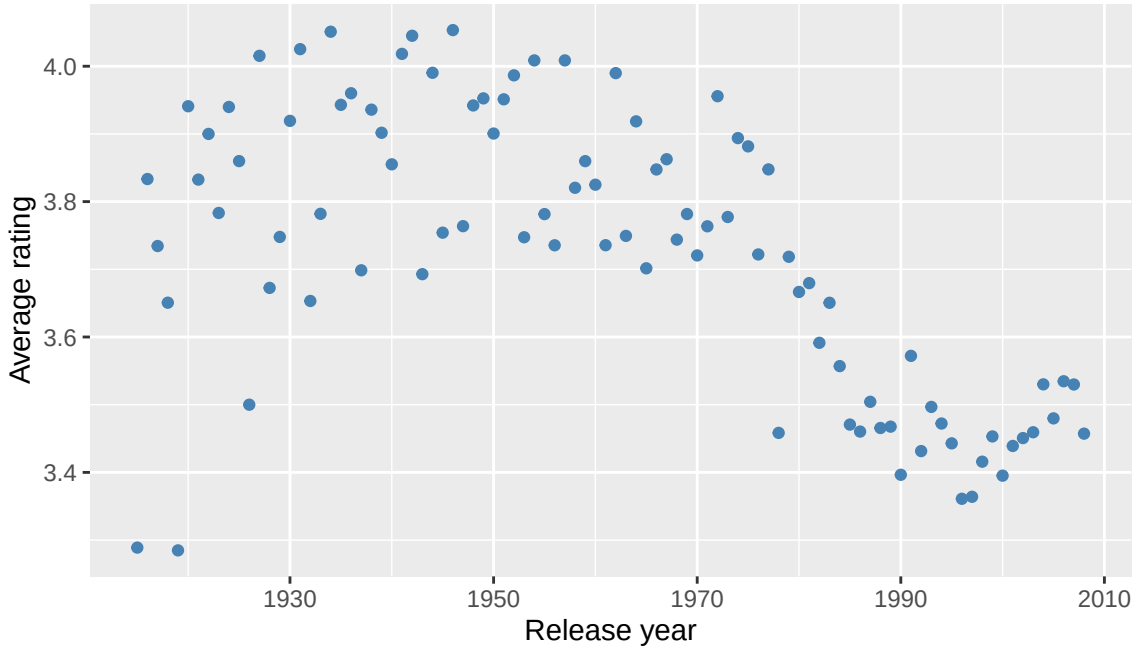
2.2.2 Number of ratings per movie and per user





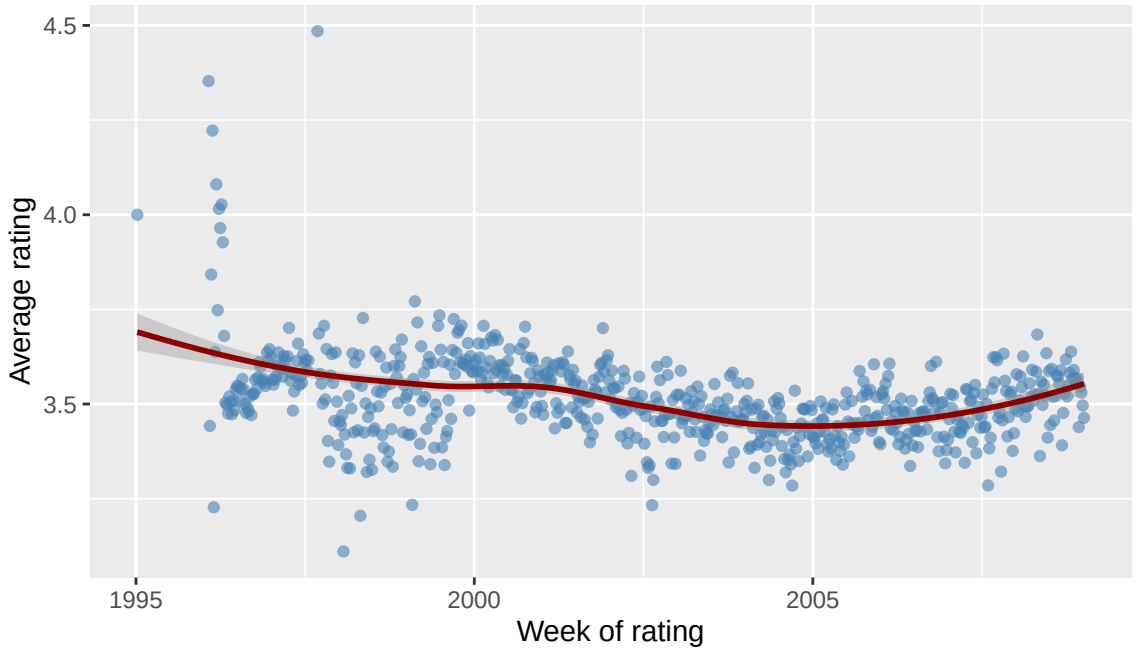
The matrix is highly sparse: only about 1.2% of the possible user-movie pairs have a rating. Most movies have between 30 and 300 ratings, but some have only one, while blockbusters have more than 30,000. On the user side, most users rated between 20 and 100 movies, but a few rated more than 1,000. This imbalance shows that averages based on few observations are not reliable, which justifies the use of regularization in the models.

2.2.3 Release year



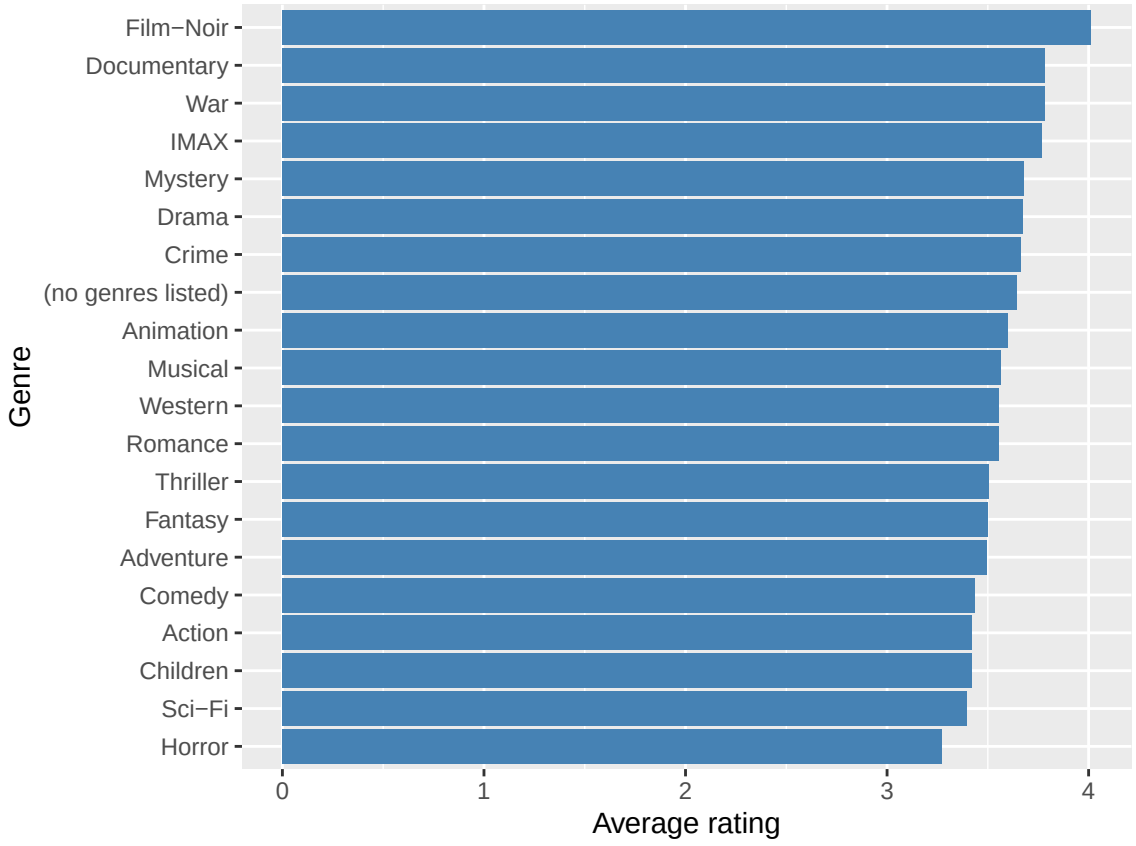
Movies released between 1930 and 1970 have higher average ratings (3.8 to 4.0). From about 1975 there is a decline, reaching about 3.4 in the 1990s, followed by a slight recovery after 2000. This pattern suggests a survivorship bias: mostly classic old movies are still watched and rated today.

2.2.4 Week of rating



The effect of the rating date is weak, with a small variation (about 3.45 to 3.65). There is a slight decline until about 2005 and a recovery afterwards. This analysis was inspired by the exercises of Chapter 33 of the course textbook, and it suggests that a rating week effect may bring only modest gains.

2.2.5 Genres



There are clear differences between genres: Film-Noir (about 4.0) and Documentary (about 3.8) have the highest averages, while Horror (about 3.3) and Sci-Fi (about 3.4) have the lowest. Niche genres tend to receive higher ratings, possibly because of a selection bias: their audience is already predisposed to like them.

2.3 Modeling approach

Models were evaluated with the root mean squared error:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{u,i}\left(\hat{y}_{u,i} - y_{u,i}\right)^2}$$

2.3.1 Baseline and bias effects

The models were built step by step. The most complete bias model is:

$$Y_{u,i} = \mu + b_i + b_u + b_y + b_g + b_t + \varepsilon_{u,i}$$

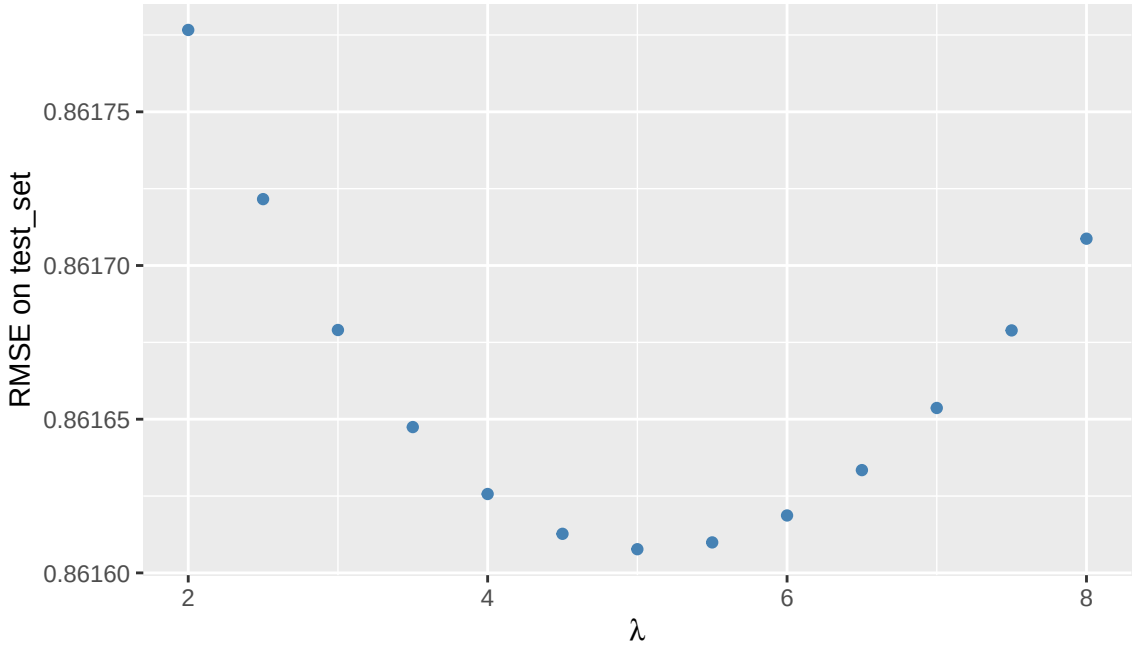
where μ is the overall average, b_i the movie effect, b_u the user effect, b_y the release year effect, b_g the genre effect and b_t the rating week effect. Each effect is estimated from what the previous effects do not explain.

- **Model 1, overall average:** RMSE = 1.0593 (baseline).
- **Model 2, movie effect:** RMSE = 0.9413 (an improvement of about 11%).
- **Model 3, movie + user effects:** RMSE = 0.8629 (already below the target on the `test_set`).
- **Models 4 to 6, release year, genre and rating week effects:** small gains (RMSE = 0.8622 with all effects), consistent with the data exploration. Following the idea of the textbook exercises, each genre combination (for example "Action|Comedy") was treated as one category.

2.3.2 Regularization

Each effect is estimated as the sum of residuals divided by the number of ratings plus λ. For the movie effect:

$$\hat{b}_i(\lambda) = \frac{1}{n_i + \lambda} \sum_{u=1}^{n_i} (Y_{u,i} - \hat{\mu})$$



Regularization was applied to all five effects (movie, user, release year, genre and rating week), while the course textbook regularizes only the movie and user effects. The best value was $\lambda = 5$, which reduced the RMSE to 0.8616. This prevents distortions caused by effects estimated from few ratings.

2.3.3 Matrix factorization of the residuals

$$Y_{u,i} = \mu + b_i + b_u + b_y + b_g + b_t + p_u^\top q_i + \varepsilon_{u,i}$$

Matrix factorization was applied to the residuals of the regularized model. Section 33.11 of the course textbook describes this model but states that fitting it is beyond the scope of the book; here it was fitted with the `recosystem` package. The hyperparameters were tuned with 3-fold cross-validation on the training residuals (best: 30 latent factors, learning rate 0.1). This step brought the largest improvement: RMSE = 0.7899, showing that user-movie interactions explain much more than isolated effects.

3 Results

Table 1: RMSE of each model (models 1 to 8 on test_set; final model on final_holdout_test)

Model	RMSE
Overall average	1.05927
Movie effect model	0.94132
Movie + user effect model	0.86293
Movie + user + release year effect model	0.86261
Movie + user + release year + genre effect model	0.86236
Movie + user + release year + genre + rating date effect model	0.86218
Regularized movie + user + release year + genre + rating date effect model	0.86161
Regularized effects + matrix factorization model	0.78990
Final model (trained on edx, evaluated on final_holdout_test)	0.78655

The results show the step-by-step evolution of the models. The baseline (overall average) had an RMSE of about 1.059, serving as the initial reference. Adding the movie effect reduced the error to 0.941, an improvement of about 11%. With the user effect, the RMSE reached 0.8629, already below the target of 0.8649.

Adding the release year, genre and rating week effects brought small gains. This happens because these factors were already partially captured by the movie effect: each movie has a single release year, genres are intrinsic characteristics of the movie, and the rating date showed only a weak effect in the data exploration.

Regularization reduced the RMSE to 0.8616. The gain was small but consistent, and it makes the estimates more stable for movies and users with few ratings.

The largest improvement came from matrix factorization, which captured interactions between users and movies, such as the preference of some users for certain types of movies. This model reduced the RMSE to 0.7899, an improvement of more than 8% compared with the regularized model.

Finally, the final model, trained on the whole `edx` set and evaluated only once on the `final_holdout_test` set, achieved an RMSE of **0.78655**. This value is slightly better than the one obtained on the `test_set` (0.7899), thanks to the use of more training data, and it confirms that the model generalizes well, with no signs of overfitting.

4 Conclusion

This project showed that it is possible to build an effective recommendation system from sparse data by combining simple effects (overall average, movie and user) with more advanced techniques (regularization and matrix factorization). The final model exceeded the target, reaching an RMSE of 0.78655 on the `final_holdout_test` set.

Limitations. The release year, genre and rating date effects had only a small impact. Matrix factorization depends on careful tuning of hyperparameters and on computational resources: because of time and memory limits, only a reduced grid of hyperparameters was tested, and since the factorization is trained in parallel, its results vary slightly (in the fourth decimal place) between runs. In addition, the model cannot make good predictions for new users or new movies without previous ratings (the "cold start" problem).

Future work. Possible next steps include exploring hybrid models that combine content information (such as genres and titles) with collaborative filtering, testing neural network approaches to matrix factorization, testing a larger grid of hyperparameters, and evaluating the scalability of the approach on larger and more recent versions of the MovieLens dataset.

5 References

- Irizarry, R. A. *Introduction to Data Science: Data Analysis and Prediction Algorithms with R*. Chapter 33 (Large datasets: recommendation systems, regularization and matrix factorization).
- Harper, F. M. and Konstan, J. A. (2015). The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems*, 5(4). Dataset: https://grouplens.org/datasets/movielens/10m/
- Qiu, Y. *recosystem: Recommender System Using Parallel Matrix Factorization*. R package. https://CRAN.R-project.org/package=recosystem
- HarvardX PH125.9x, Data Science: Capstone. Course-provided code for creating the `edx` and `final_holdout_test` sets.
- Anthropic. Claude (generative AI tool), used during this project as described in the note on sources at the top of `movielens.R`.